

GridBalance: Complete Technical Research Dossier

Exhaustive Repository Audit, Data Quality, ML Experiments, Robustness & Production Deployment

GridBalance — Real ML Inference Backend Report

1. System Architecture

The GridBalance Machine Learning Inference Backend is built using **FastAPI** to serve real-time predictions from both frozen ML research engines:

- 1. **SGCC Meter Tampering Classifier:** 18-feature XGBoost Pipeline fitted with a median imputer for handling missing feature values, operating at a frozen decision threshold of **0.50**.
- 2. **UCI Electricity Load Forecaster:** 29-feature XGBoost Regressor for next-hour ($t+1$) aggregate grid load forecasting.

Directory & Package Structure

gridbalance/backend/	
■■■ main.py	# FastAPI entry point & lifespan model loader
■■■ config.py	# Absolute model paths & frozen benchmark metrics
■■■ api/	
■ ■■■ detection.py	# POST /api/v1/detection/analyze & POST /api/v1/detection/batch
■ ■■■ forecasting.py	# POST /api/v1/forecast/next-hour
■ ■■■ models.py	# GET /api/v1/models
■ ■■■ health.py	# GET /api/v1/health
■■■ services/	
■ ■■■ sgcc_service.py	# SGCC inference service & CSV batch parser
■ ■■■ forecasting_service.py	# UCI next-hour forecasting service
■■■ inference/	
■ ■■■ sgcc_predictor.py	# Date sorting, missingness audit, 18-feature extraction, XGBoost inference
■ ■■■ uci_forecaster.py	# Lag, rolling, calendar feature extraction, XGBoost regression
■■■ schemas/	
■■■ sgcc.py	# Pydantic schemas for SGCC request/response
■■■ forecasting.py	# Pydantic schemas for UCI load forecast
■■■ models.py	# Model metadata response schema
■■■ health.py	# System health response schema

2. Frozen Model Artifacts Used

Both models were loaded ONCE at application startup and remain strictly frozen without any retraining, retuning, or feature definition changes.

- 1. **SGCC Classifier Model:**
 - Path: models/sgcc_tuned/xgboost_reduced_best.joblib

- Model Type: `sklearn.pipeline.Pipeline (SimpleImputer(strategy='median') + XGBClassifier)`
- Features (18): ['missing_streak_count', 'monthly_std_4', 'monthly_std_3', 'mean_abs_daily_change', 'missing_count', 'monthly_mean_10', 'longest_missing_streak', 'peak_to_average_ratio', 'monthly_std_5', 'monthly_std_10', 'monthly_std_11', 'cv', 'monthly_mean_8', 'max', 'monthly_std_8', 'monthly_mean_11', 'monthly_std_1', 'monthly_mean_2']
- Decision Threshold: **0.50**
- Sealed Test Metrics: F1 = 0.4014, PR-AUC = 0.4049, ROC-AUC = 0.8347

2. UCI Forecasting Model:

- Path: `models/uci_forecasting/xgb_forecasting_frozen.joblib`
- Model Type: `xgboost.sklearn.XGBRegressor`
- Features (29): 10 Lags (`load_t ... load_t_minus_168`), 10 Rolling Statistics (`rolling_mean_3 ... rolling_max_24`), 9 Calendar & Cyclic features (`hour, day_of_week, sin_hour, cos_hour, etc.`).
- Forecast Horizon: Single step next-hour ($t+1$)
- Sealed Test Metrics: MAE = 4,421.55 kWh, RMSE = 6,654.94 kWh, $R^2 = 0.9944$, sMAPE = 2.18%

3. Endpoints & Schemas

A. Health Check: GET `/api/v1/health`

- **Description:** Verifies API operation and confirms both frozen models are loaded into memory.
- **Output Schema:**

```
{
  "status": "healthy",
  "sgcc_model": "loaded",
  "forecast_model": "loaded",
  "timestamp": "2026-09-28T02:46:10.327084"
}
```

B. Model Information: GET `/api/v1/models`

- **Description:** Exposes frozen model metadata and research evaluation benchmarks on sealed test sets.

C. Single Meter Detection: POST `/api/v1/detection/analyze`

- **Input Payload:** `meter_id` (string), `dates` (List[string]), `consumption` (List[float]).
- **Pipeline:**
 1. Chronological date parsing & sorting.
 2. Missingness auditing (`missing_count, missing_ratio, longest_missing_streak, missing_streak_count`).
 3. Bounded forward-fill (`limit=7`).
 4. Exact 18-feature extraction.
 5. Fitted median imputation & XGBoost probability prediction.
 6. Thresholding at 0.50.

- **Terminology Rule:** Uses "potential_tampering", NEVER "confirmed_theft".

D. Batch Meter Detection: POST /api/v1/detection/batch

- **Input:** CSV file upload (UploadFile).
- **Processing:** Evaluates each meter independently. Supports JSON response or CSV export via query parameter `export_csv=True`.

E. UCI Next-Hour Forecast: POST /api/v1/forecast/next-hour

- **Input Payload:** history (List of timestamp & load_kwh points, minimum 168 hours).
- **Pipeline:**
 1. Chronological timestamp sorting.
 2. 29 lag, rolling, and calendar feature extraction at time t .
 3. XGBoost regression prediction for $t+1$.

4. Automated Test Suite

A complete test suite is implemented in `tests/` covering:

- `test_model_loading.py`: Verifies frozen model artifacts exist and load correctly.
- `test_sgcc_inference.py`: Tests chronological date sorting, missingness handling, bounded ffill, risk score calculation, and strict terminology enforcement (`potential_tampering`).
- `test_uci_forecasting.py`: Tests 29-feature generation, minimum history validation (≥ 168 hours), and next-hour target timestamp calculation.
- `test_api_endpoints.py`: Integration testing using FastAPI `TestClient` for all endpoints.

Test Execution Results:

```
collected 13 items
tests/test_api_endpoints.py::test_health_endpoint PASSED
tests/test_api_endpoints.py::test_models_info_endpoint PASSED
tests/test_api_endpoints.py::test_detection_analyze_endpoint PASSED
tests/test_api_endpoints.py::test_detection_invalid_dates PASSED
tests/test_api_endpoints.py::test_forecasting_endpoint PASSED
tests/test_api_endpoints.py::test_forecasting_insufficient_history PASSED
tests/test_model_loading.py::test_sgcc_model_loading PASSED
tests/test_model_loading.py::test_uci_model_loading PASSED
tests/test_sgcc_inference.py::test_chronological_sorting PASSED
tests/test_sgcc_inference.py::test_sgcc_inference_output PASSED
tests/test_sgcc_inference.py::test_sgcc_tampering_pattern PASSED
tests/test_uci_forecasting.py::test_uci_forecasting_valid PASSED
tests/test_uci_forecasting.py::test_uci_forecasting_insufficient_history PASSED

===== 13 passed in 1.81s =====
```

5. End-to-End Verification

The running FastAPI server was verified end-to-end against real development input:

- GET `http://127.0.0.1:8000/api/v1/health` → 200 OK, status: healthy.
 - POST `http://127.0.0.1:8000/api/v1/detection/analyze` → 200 OK, returned real probability (0.1229), risk level (Low), data quality metrics, top SHAP-aligned feature importances, and cleaned consumption history.
 - POST `http://127.0.0.1:8000/api/v1/forecast/next-hour` → 200 OK, returned `predicted_load_kwh`: 142962.55 for next hour ($t+1$).
-

6. System Limitations

1. **Forecasting Horizon:** The UCI forecasting model is strictly a single-step next-hour ($t+1$) model. Multi-hour recursive forecasting is not claimed or supported by this frozen regressor.
2. **Minimum History Requirement:** UCI forecasting requires at least 168 hours (7 days) of historical hourly load observations to populate the 168-hour lag and rolling windows.
3. **Threshold Stability:** The SGCC decision threshold is fixed at 0.50 as established during the research phase.